

```

#!/usr/bin/env python3
"""
Innovara Technology Database System
Complete implementation of the technology catalog and search engine
for the autonomous R&D platform
"""

import json
import sqlite3
import datetime
import re
from typing import Dict, List, Any, Optional, Tuple
from dataclasses import dataclass, asdict
from enum import Enum
import difflib
from collections import defaultdict
import math

class TechnologyCategory(Enum):
    HARDWARE = "hardware"
    SOFTWARE = "software"
    FRAMEWORK = "framework"
    LANGUAGE = "language"
    DATABASE = "database"
    CLOUD_SERVICE = "cloud_service"
    API = "api"
    PROTOCOL = "protocol"
    ALGORITHM = "algorithm"
    TOOL = "tool"
    PLATFORM = "platform"
    LIBRARY = "library"
    OPERATING_SYSTEM = "operating_system"
    BLOCKCHAIN = "blockchain"
    AI_ML = "ai_ml"
    MOBILE = "mobile"
    WEB = "web"
    SECURITY = "security"
    NETWORKING = "networking"
    STORAGE = "storage"

class MaturityLevel(Enum):
    EXPERIMENTAL = "experimental"
    ALPHA = "alpha"
    BETA = "beta"
    STABLE = "stable"
    MATURE = "mature"
    LEGACY = "legacy"
    DEPRECATED = "deprecated"

```

```

class LicenseType(Enum):
    OPEN_SOURCE = "open_source"
    PROPRIETARY = "proprietary"
    FREEMIUM = "freemium"
    COMMERCIAL = "commercial"
    MIT = "mit"
    GPL = "gpl"
    APACHE = "apache"
    BSD = "bsd"

@dataclass
class Technology:
    """Represents a single technology component"""
    id: str
    name: str
    category: TechnologyCategory
    description: str
    version: str
    release_date: str
    maturity_level: MaturityLevel
    license_type: LicenseType
    vendor: str
    website: str
    repository: str
    documentation: str
    api_endpoints: List[str]
    dependencies: List[str]
    compatible_with: List[str]
    programming_languages: List[str]
    platforms: List[str]
    use_cases: List[str]
    performance_metrics: Dict[str, Any]
    cost_model: Dict[str, Any]
    security_features: List[str]
    scalability_limits: Dict[str, Any]
    integration_complexity: int # 1-10 scale
    learning_curve: int # 1-10 scale
    community_size: int
    github_stars: int
    last_updated: str
    tags: List[str]

@dataclass
class EvolutionPattern:
    """Represents technology evolution patterns"""
    pattern_id: str
    name: str

```

```
description: str
stages: List[str]
timeline: List[str]
progression_indicators: List[str]
next_stage_prediction: str
confidence_score: float
```

```
class InnovaraTechnologyDatabase:
```

```
    """Main database system for Innovara technology catalog"""
```

```
    def __init__(self, db_path: str = "innovara_technology_database.db"):
        self.db_path = db_path
        self.init_database()
        self.load_historical_timeline()
        self.load_modern_technologies()
        self.build_evolution_patterns()
```

```
    def init_database(self):
```

```
        """Initialize SQLite database with all necessary tables"""
```

```
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
```

```
        # Technologies table
```

```
        cursor.execute("""
```

```
            CREATE TABLE IF NOT EXISTS technologies (
```

```
                id TEXT PRIMARY KEY,
                name TEXT NOT NULL,
                category TEXT NOT NULL,
                description TEXT,
                version TEXT,
                release_date TEXT,
                maturity_level TEXT,
                license_type TEXT,
                vendor TEXT,
                website TEXT,
                repository TEXT,
                documentation TEXT,
                api_endpoints TEXT,
                dependencies TEXT,
                compatible_with TEXT,
                programming_languages TEXT,
                platforms TEXT,
                use_cases TEXT,
                performance_metrics TEXT,
                cost_model TEXT,
                security_features TEXT,
                scalability_limits TEXT,
                integration_complexity INTEGER,
```

```

        learning_curve INTEGER,
        community_size INTEGER,
        github_stars INTEGER,
        last_updated TEXT,
        tags TEXT
    )
    """)

# Evolution Patterns table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS evolution_patterns (
        pattern_id TEXT PRIMARY KEY,
        name TEXT NOT NULL,
        description TEXT,
        stages TEXT,
        timeline TEXT,
        progression_indicators TEXT,
        next_stage_prediction TEXT,
        confidence_score REAL
    )
    """)

# Technology Relationships table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS technology_relationships (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        source_tech_id TEXT,
        target_tech_id TEXT,
        relationship_type TEXT,
        strength REAL,
        description TEXT,
        FOREIGN KEY (source_tech_id) REFERENCES technologies(id),
        FOREIGN KEY (target_tech_id) REFERENCES technologies(id)
    )
    """)

# Innovation Opportunities table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS innovation_opportunities (
        id TEXT PRIMARY KEY,
        title TEXT NOT NULL,
        description TEXT,
        technology_combination TEXT,
        market_potential INTEGER,
        technical_feasibility INTEGER,
        development_effort INTEGER,
        risk_level INTEGER,
        estimated_timeline INTEGER,

```

```

        required_skills TEXT,
        potential_revenue TEXT,
        competitive_advantage TEXT,
        created_date TEXT
    )
    """)

```

```

conn.commit()
conn.close()

```

```

def load_historical_timeline(self):

```

```

    """Load historical technology timeline from Pascal's Calculator to AI"""

```

```

    historical_techs = [

```

```

        Technology(

```

```

            id="pascal_calculator_1642",
            name="Pascal's Calculator (Pascaline)",
            category=TechnologyCategory.HARDWARE,
            description="First mechanical calculator invented by Blaise Pascal",
            version="1.0",
            release_date="1642-06-19",
            maturity_level=MaturityLevel.LEGACY,
            license_type=LicenseType.PROPRIETARY,
            vendor="Blaise Pascal",
            website="",
            repository="",
            documentation="Historical documents",
            api_endpoints=[],
            dependencies=[],
            compatible_with=[],
            programming_languages=[],
            platforms=["Mechanical"],
            use_cases=["Mathematical calculations", "Accounting"],
            performance_metrics={"accuracy": "High for basic arithmetic"},
            cost_model={"initial_cost": "Very high for the era"},
            security_features=[],
            scalability_limits={"max_operations": "Basic arithmetic only"},
            integration_complexity=1,
            learning_curve=3,
            community_size=0,
            github_stars=0,
            last_updated="1642-06-19",
            tags=["historical", "mechanical", "calculator", "arithmetic"]

```

```

        ),

```

```

        Technology(

```

```

            id="transistor_1947",
            name="Transistor",
            category=TechnologyCategory.HARDWARE,
            description="Semiconductor device for amplifying or switching electronic signals",

```

```

version="Point-contact",
release_date="1947-12-23",
maturity_level=MaturityLevel.MATURE,
license_type=LicenseType.OPEN_SOURCE,
vendor="Bell Labs",
website="",
repository="",
documentation="Bell Labs technical papers",
api_endpoints=[],
dependencies=[],
compatible_with=["All electronic devices"],
programming_languages=[],
platforms=["Electronic circuits"],
use_cases=["Signal amplification", "Digital switching", "Computing"],
performance_metrics={"switching_speed": "Nanoseconds", "power_efficiency": "High"},
cost_model={"manufacturing_cost": "Low at scale"},
security_features=[],
scalability_limits={"miniaturization": "Atomic level"},
integration_complexity=8,
learning_curve=9,
community_size=1000000,
github_stars=0,
last_updated="2024-01-01",
tags=["semiconductor", "electronics", "computing", "foundation"]
),
Technology(
    id="ai_term_1955",
    name="Artificial Intelligence (Term Coined)",
    category=TechnologyCategory.AI_ML,
    description="Term 'Artificial Intelligence' coined by John McCarthy at Dartmouth Conference",
    version="Conceptual",
    release_date="1955-01-01",
    maturity_level=MaturityLevel.MATURE,
    license_type=LicenseType.OPEN_SOURCE,
    vendor="John McCarthy",
    website="",
    repository="",
    documentation="Dartmouth Conference papers",
    api_endpoints=[],
    dependencies=["Computing", "Mathematics", "Logic"],
    compatible_with=["All computing platforms"],
    programming_languages=["LISP", "Prolog", "Python"],
    platforms=["Academic", "Research", "Commercial"],
    use_cases=["Machine learning", "Expert systems", "Natural language processing"],
    performance_metrics={"intelligence_level": "Approaching human-level"},
    cost_model={"research_cost": "Billions invested"},
    security_features=["AI safety", "Alignment research"],
    scalability_limits={"compute_requirements": "Exponential growth"},

```

```

        integration_complexity=10,
        learning_curve=10,
        community_size=10000000,
        github_stars=0,
        last_updated="2024-01-01",
        tags=["ai", "machine-learning", "intelligence", "foundational"]
    )
]

```

Add modern AI technologies

```

modern_ai_techs = [
    Technology(
        id="chatgpt_2022",
        name="ChatGPT",
        category=TechnologyCategory.AI_ML,
        description="Large language model chatbot developed by OpenAI",
        version="GPT-4",
        release_date="2022-11-30",
        maturity_level=MaturityLevel.STABLE,
        license_type=LicenseType.COMMERCIAL,
        vendor="OpenAI",
        website="https://chat.openai.com",
        repository="",
        documentation="https://platform.openai.com/docs",
        api_endpoints=["https://api.openai.com/v1/chat/completions"],
        dependencies=["GPT architecture", "Transformer models"],
        compatible_with=["REST APIs", "Python", "JavaScript"],
        programming_languages=["Python", "JavaScript", "cURL"],
        platforms=["Web", "API", "Mobile"],
        use_cases=["Conversational AI", "Content generation", "Code assistance"],
        performance_metrics={"response_time": "1-3 seconds", "accuracy": "High"},
        cost_model={"pricing": "Pay-per-token", "free_tier": "Limited"},
        security_features=["Content filtering", "Usage monitoring"],
        scalability_limits={"rate_limits": "API dependent"},
        integration_complexity=3,
        learning_curve=2,
        community_size=100000000,
        github_stars=0,
        last_updated="2024-01-01",
        tags=["llm", "chatbot", "openai", "gpt", "conversational-ai"]
    ),
    Technology(
        id="claude_3_2024",
        name="Claude 3",
        category=TechnologyCategory.AI_ML,
        description="Advanced AI assistant by Anthropic with constitutional AI training",
        version="3.5 Sonnet",
        release_date="2024-03-04",

```

```

        maturity_level=MaturityLevel.STABLE,
        license_type=LicenseType.COMMERCIAL,
        vendor="Anthropic",
        website="https://claude.ai",
        repository="",
        documentation="https://docs.anthropic.com",
        api_endpoints=["https://api.anthropic.com/v1/messages"],
        dependencies=["Constitutional AI", "Transformer architecture"],
        compatible_with=["REST APIs", "Python", "JavaScript"],
        programming_languages=["Python", "JavaScript", "cURL"],
        platforms=["Web", "API", "Mobile"],
        use_cases=["AI assistance", "Analysis", "Creative writing", "Code review"],
        performance_metrics={"context_length": "200K tokens", "safety": "High"},
        cost_model={"pricing": "Pay-per-token", "free_tier": "Available"},
        security_features=["Constitutional AI", "Harmlessness training"],
        scalability_limits={"context_window": "200K tokens"},
        integration_complexity=3,
        learning_curve=2,
        community_size=10000000,
        github_stars=0,
        last_updated="2024-03-04",
        tags=["llm", "anthropic", "constitutional-ai", "safety", "assistant"]
    )
]

```

```

# Store all technologies
all_technologies = historical_techs + modern_ai_techs
self.store_technologies(all_technologies)

```

```

def load_modern_technologies(self):
    """Load modern technology stack"""
    modern_techs = [
        Technology(
            id="python_3_12",
            name="Python",
            category=TechnologyCategory.LANGUAGE,
            description="High-level, interpreted programming language with dynamic semantics",
            version="3.12",
            release_date="2023-10-02",
            maturity_level=MaturityLevel.MATURE,
            license_type=LicenseType.OPEN_SOURCE,
            vendor="Python Software Foundation",
            website="https://python.org",
            repository="https://github.com/python/cpython",
            documentation="https://docs.python.org",
            api_endpoints=["https://pypi.org/simple/"],
            dependencies=["C compiler", "Standard library"],
            compatible_with=["All major platforms"],

```



```

        programming_languages=["Python"],
        platforms=["Windows", "macOS", "Linux", "Unix"],
        use_cases=["Web development", "Data science", "AI/ML", "Automation"],
        performance_metrics={"execution_speed": "Interpreted", "memory_usage": "Moderate"},
        cost_model={"license_cost": "Free", "development_cost": "Low"},
        security_features=["Sandboxing", "Type hints", "Security advisories"],
        scalability_limits={"gil_limitation": "Single-threaded execution"},
        integration_complexity=3,
        learning_curve=4,
        community_size=15000000,
        github_stars=62000,
        last_updated="2024-01-01",
        tags=["programming", "interpreted", "versatile", "popular"]
    ),
    Technology(
        id="aws_2024",
        name="Amazon Web Services",
        category=TechnologyCategory.CLOUD_SERVICE,
        description="Comprehensive cloud computing platform",
        version="2024",
        release_date="2006-03-14",
        maturity_level=MaturityLevel.MATURE,
        license_type=LicenseType.COMMERCIAL,
        vendor="Amazon",
        website="https://aws.amazon.com",
        repository="https://github.com/aws",
        documentation="https://docs.aws.amazon.com",
        api_endpoints=["https://aws.amazon.com/api/"],
        dependencies=["Internet connectivity"],
        compatible_with=["All major platforms and languages"],
        programming_languages=["All"],
        platforms=["Cloud"],
        use_cases=["Web hosting", "Data storage", "Machine learning", "Enterprise applications"],
        performance_metrics={"uptime": "99.99%", "global_regions": "31"},
        cost_model={"pricing": "Pay-as-you-go", "free_tier": "12 months"},
        security_features=["IAM", "VPC", "Encryption", "Compliance certifications"],
        scalability_limits={"virtually_unlimited": "Auto-scaling"},
        integration_complexity=8,
        learning_curve=9,
        community_size=10000000,
        github_stars=0,
        last_updated="2024-01-01",
        tags=["cloud", "scalable", "enterprise", "comprehensive"]
    )
]

self.store_technologies(modern_techs)

```

```

def build_evolution_patterns(self):
    """Build technology evolution patterns"""
    patterns = [
        EvolutionPattern(
            pattern_id="calculator_to_ai",
            name="Calculator → Computer → AI Evolution",
            description="Evolution from mechanical calculation to artificial intelligence",
            stages=["Mechanical Calculator", "Electronic Computer", "Personal Computer", "AI Systems"],
            timeline=["1642", "1940s", "1970s", "2020s"],
            progression_indicators=["Automation level", "Processing power", "Intelligence capability"],
            next_stage_prediction="Artificial General Intelligence (AGI)",
            confidence_score=0.85
        ),
        EvolutionPattern(
            pattern_id="communication_evolution",
            name="Radio → Mobile → Internet Evolution",
            description="Evolution of communication technologies",
            stages=["Radio", "Radio Phone", "Cellular Mobile", "Smartphone", "AI-Powered Devices"],
            timeline=["1895", "1946", "1983", "2007", "2020s"],
            progression_indicators=["Mobility", "Interactivity", "Intelligence"],
            next_stage_prediction="Neural Interface Communication",
            confidence_score=0.75
        )
    ]

    self.store_evolution_patterns(patterns)

def store_technologies(self, technologies: List[Technology]):
    """Store technologies in database"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    for tech in technologies:
        cursor.execute("""
            INSERT OR REPLACE INTO technologies VALUES (
                ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?
            )
        """, (
            tech.id, tech.name, tech.category.value, tech.description, tech.version,
            tech.release_date, tech.maturity_level.value, tech.license_type.value,
            tech.vendor, tech.website, tech.repository, tech.documentation,
            json.dumps(tech.api_endpoints), json.dumps(tech.dependencies),
            json.dumps(tech.compatible_with), json.dumps(tech.programming_languages),
            json.dumps(tech.platforms), json.dumps(tech.use_cases),
            json.dumps(tech.performance_metrics), json.dumps(tech.cost_model),
            json.dumps(tech.security_features), json.dumps(tech.scalability_limits),
            tech.integration_complexity, tech.learning_curve, tech.community_size,
            tech.github_stars, tech.last_updated, json.dumps(tech.tags)
        ))

```

```

    ))

    conn.commit()
    conn.close()

def store_evolution_patterns(self, patterns: List[EvolutionPattern]):
    """Store evolution patterns in database"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    for pattern in patterns:
        cursor.execute("""
            INSERT OR REPLACE INTO evolution_patterns VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            pattern.pattern_id, pattern.name, pattern.description,
            json.dumps(pattern.stages), json.dumps(pattern.timeline),
            json.dumps(pattern.progression_indicators),
            pattern.next_stage_prediction, pattern.confidence_score
        ))

    conn.commit()
    conn.close()

def search_technologies(self, query: str, limit: int = 20) -> List[Dict]:
    """Search technologies with fuzzy matching"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    # Fuzzy search across name, description, and tags
    cursor.execute("""
        SELECT * FROM technologies
        WHERE name LIKE ? OR description LIKE ? OR tags LIKE ?
        ORDER BY
            CASE
                WHEN name LIKE ? THEN 1
                WHEN description LIKE ? THEN 2
                ELSE 3
            END,
            github_stars DESC
        LIMIT ?
    """, (f'%{query}%', f'%{query}%', f'%{query}%', f'%{query}%', f'%{query}%', limit))

    results = cursor.fetchall()
    conn.close()

    # Convert to dictionaries
    columns = [
        'id', 'name', 'category', 'description', 'version', 'release_date',

```

```

        'maturity_level', 'license_type', 'vendor', 'website', 'repository',
        'documentation', 'api_endpoints', 'dependencies', 'compatible_with',
        'programming_languages', 'platforms', 'use_cases', 'performance_metrics',
        'cost_model', 'security_features', 'scalability_limits',
        'integration_complexity', 'learning_curve', 'community_size',
        'github_stars', 'last_updated', 'tags'
    ]

    return [dict(zip(columns, row)) for row in results]

def find_compatible_technologies(self, tech_id: str) -> List[Dict]:
    """Find technologies compatible with given technology"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    # Get the technology
    cursor.execute('SELECT compatible_with FROM technologies WHERE id = ?', (tech_id,))
    result = cursor.fetchone()

    if not result:
        return []

    compatible_with = json.loads(result[0] or '[]')

    # Find technologies that match compatibility
    compatible_techs = []
    for compat in compatible_with:
        cursor.execute("""
            SELECT * FROM technologies
            WHERE name LIKE ? OR tags LIKE ?
            """, (f'%{compat}%', f'%{compat}%'))
        compatible_techs.extend(cursor.fetchall())

    conn.close()

    # Convert to dictionaries and remove duplicates
    columns = [
        'id', 'name', 'category', 'description', 'version', 'release_date',
        'maturity_level', 'license_type', 'vendor', 'website', 'repository',
        'documentation', 'api_endpoints', 'dependencies', 'compatible_with',
        'programming_languages', 'platforms', 'use_cases', 'performance_metrics',
        'cost_model', 'security_features', 'scalability_limits',
        'integration_complexity', 'learning_curve', 'community_size',
        'github_stars', 'last_updated', 'tags'
    ]

    unique_techs = {}
    for row in compatible_techs:

```

```

        tech_dict = dict(zip(columns, row))
        unique_techs[tech_dict['id']] = tech_dict

    return list(unique_techs.values())

def analyze_technology_trends(self) -> Dict[str, Any]:
    """Analyze technology trends and patterns"""
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    # Category distribution
    cursor.execute("""
        SELECT category, COUNT(*) as count
        FROM technologies
        GROUP BY category
        ORDER BY count DESC
    """)
    category_distribution = dict(cursor.fetchall())

    # Maturity levels
    cursor.execute("""
        SELECT maturity_level, COUNT(*) as count
        FROM technologies
        GROUP BY maturity_level
        ORDER BY count DESC
    """)
    maturity_distribution = dict(cursor.fetchall())

    # Top technologies by community size
    cursor.execute("""
        SELECT name, community_size, github_stars
        FROM technologies
        WHERE community_size > 0
        ORDER BY community_size DESC
        LIMIT 10
    """)
    top_technologies = cursor.fetchall()

    # Recent technologies (last 5 years)
    cursor.execute("""
        SELECT name, release_date, category
        FROM technologies
        WHERE release_date >= '2019-01-01'
        ORDER BY release_date DESC
    """)
    recent_technologies = cursor.fetchall()

    conn.close()

```

```

return {
    "category_distribution": category_distribution,
    "maturity_distribution": maturity_distribution,
    "top_technologies": top_technologies,
    "recent_technologies": recent_technologies,
    "total_technologies": sum(category_distribution.values())
}

def predict_next_evolution(self, current_tech: str) -> Dict[str, Any]:
    """Predict next evolution stage for a technology"""
    # Find evolution patterns that include this technology
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM evolution_patterns")
    patterns = cursor.fetchall()

    predictions = []
    for pattern in patterns:
        stages = json.loads(pattern[3]) # stages column
        if current_tech.lower() in [stage.lower() for stage in stages]:
            current_index = next(i for i, stage in enumerate(stages) if current_tech.lower() in stage.lower())
            if current_index < len(stages) - 1:
                next_stage = stages[current_index + 1]
                predictions.append({
                    "pattern_name": pattern[1],
                    "current_stage": stages[current_index],
                    "next_stage": next_stage,
                    "confidence": pattern[7] # confidence_score column
                })

    conn.close()

    return {
        "current_technology": current_tech,
        "predictions": predictions,
        "recommendation": predictions[0] if predictions else None
    }

def generate_innovation_opportunities(self, tech_combination: List[str]) -> Dict[str, Any]:
    """Generate innovation opportunities from technology combinations"""
    opportunities = []

    for i, tech1 in enumerate(tech_combination):
        for tech2 in tech_combination[i+1:]:
            # Find technologies in database
            tech1_data = self.search_technologies(tech1, 1)

```

```

tech2_data = self.search_technologies(tech2, 1)

if tech1_data and tech2_data:
    opportunity = {
        "id": f"innov_{tech1.lower()}_{tech2.lower()}",
        "title": f"Integration of {tech1_data[0]['name']} and {tech2_data[0]['name']}",
        "description": f"Combining {tech1} capabilities with {tech2} features",
        "technology_combination": [tech1, tech2],
        "market_potential": self._calculate_market_potential(tech1_data[0], tech2_data[0]),
        "technical_feasibility": self._calculate_technical_feasibility(tech1_data[0], tech2_data[0]),
        "development_effort": self._calculate_development_effort(tech1_data[0], tech2_data[0]),
        "risk_level": self._calculate_risk_level(tech1_data[0], tech2_data[0]),
        "estimated_timeline": "6-18 months",
        "competitive_advantage": "First-to-market integration"
    }
    opportunities.append(opportunity)

return {
    "input_technologies": tech_combination,
    "opportunities": opportunities,
    "total_opportunities": len(opportunities)
}

def _calculate_market_potential(self, tech1: Dict, tech2: Dict) -> int:
    """Calculate market potential score (1-100)"""
    # Simple scoring based on community size and maturity
    score1 = min(tech1.get('community_size', 0) / 100000, 50)
    score2 = min(tech2.get('community_size', 0) / 100000, 50)
    return int(score1 + score2)

def _calculate_technical_feasibility(self, tech1: Dict, tech2: Dict) -> int:
    """Calculate technical feasibility score (1-100)"""
    # Based on integration complexity and maturity
    complexity1 = tech1.get('integration_complexity', 5)
    complexity2 = tech2.get('integration_complexity', 5)
    avg_complexity = (complexity1 + complexity2) / 2
    return int(100 - (avg_complexity * 8)) # Lower complexity = higher feasibility

def _calculate_development_effort(self, tech1: Dict, tech2: Dict) -> int:
    """Calculate development effort score (1-100)"""
    # Based on learning curve and complexity
    learning1 = tech1.get('learning_curve', 5)
    learning2 = tech2.get('learning_curve', 5)
    avg_learning = (learning1 + learning2) / 2
    return int(avg_learning * 10)

def _calculate_risk_level(self, tech1: Dict, tech2: Dict) -> int:
    """Calculate risk level score (1-100)"""

```

```

# Based on maturity levels
maturity_scores = {
    'experimental': 80,
    'alpha': 60,
    'beta': 40,
    'stable': 20,
    'mature': 10,
    'legacy': 30,
    'deprecated': 90
}

risk1 = maturity_scores.get(tech1.get('maturity_level', 'stable'), 50)
risk2 = maturity_scores.get(tech2.get('maturity_level', 'stable'), 50)
return int((risk1 + risk2) / 2)

# Example usage and testing
if __name__ == "__main__":
    # Initialize the database
    db = InnovaraTechnologyDatabase()

    # Search for AI technologies
    ai_results = db.search_technologies("artificial intelligence")
    print(f"Found {len(ai_results)} AI technologies")

    # Analyze trends
    trends = db.analyze_technology_trends()
    print(f"Total technologies in database: {trends['total_technologies']}")
    print(f"Top category: {max(trends['category_distribution'].items(), key=lambda x: x[1])}")

    # Predict evolution
    evolution = db.predict_next_evolution("ChatGPT")
    print(f"Evolution predictions for ChatGPT: {len(evolution['predictions'])}")

    # Generate innovation opportunities
    opportunities = db.generate_innovation_opportunities(["AI", "Mobile", "Satellite"])
    print(f"Generated {opportunities['total_opportunities']} innovation opportunities")

```